

```
1 // =====
2 // TEMPLATE CRUD CONTROLLER - Copy và thay thế:
3 //   "Product"      → tên entity của bạn (Book, Fruit, Order...)
4 //   "ProductDTO"   → tên DTO tương ứng
5 //   "_context.Products" → DbSet tương ứng trong DbContext
6 // =====
7
8 using Microsoft.AspNetCore.Authorization;
9 using Microsoft.AspNetCore.Mvc;
10 using Microsoft.EntityFrameworkCore;
11 using System.ComponentModel.DataAnnotations;
12 using System.Security.Claims;
13 using YourProject.DTOs;
14 using YourProject.Models;
15
16 namespace YourProject.Controllers
17 {
18     [ApiController]
19     [Route("api/[controller]")]
20     // [Authorize]           // ← Bỏ comment nếu TOÀN BỘ controller cần  ➤
21     // [Authorize(Roles = "Admin")] // ← Bỏ comment nếu chỉ Admin mới  ➤
22     // dùng được
23     public class ProductsController : ControllerBase
24     {
25         private readonly AppDbContext _context;
26
27         public ProductsController(AppDbContext context)
28         {
29             _context = context;
30         }
31
32         // =====
33         // GET /api/products
34         // Lấy toàn bộ danh sách
35         // =====
36         [HttpGet]
37         public async Task<IActionResult> GetAll()
38         {
39             var list = await _context.Products
40                 // .Include(p => p.Category) // ← Bỏ comment nếu cần  ➤
41                 // load quan hệ
42                 .Select(p => new ProductResponseDTO
43                 {
44                     Id = p.Id,
45                     Name = p.Name,
46                     Price = p.Price,
47                     // Map thêm field nếu cần
```

```
47          //Quy tắc đơn giản: bên trái = field của DTO, bên phải = field của Model.
48      })
49      .ToListAsync();
50
51      return Ok(list);
52  }
53
54
55  // =====
56  // GET /api/products/{id}
57  // Lấy 1 bản ghi theo Id
58  // =====
59  [HttpGet("{id}")]
60  public async Task<IActionResult> GetById(int id)
61  {
62      var item = await _context.Products
63          .Include(p => p.Category) // ← Bỏ comment nếu cần load quan hệ
64          .Where(p => p.Id == id)
65          .Select(p => new ProductResponseDTO
66          {
67              Id = p.Id,
68              Name = p.Name,
69              Price = p.Price,
70          })
71          .FirstOrDefaultAsync();
72
73      if (item == null)
74          return NotFound($"Không tìm thấy Product với Id = {id}.");
75
76      return Ok(item);
77  }
78
79
80  // =====
81  // POST /api/products
82  // Tạo mới 1 bản ghi
83  // =====
84  [HttpPost]
85  // [Authorize] // ← Bỏ comment nếu endpoint này cần đăng nhập
86  // [Authorize(Roles = "Admin")] // ← Bỏ comment nếu chỉ Admin tạo được
87  public async Task<IActionResult> Create([FromBody] CreateProductDTO dto)
88  {
89      if (!ModelState.IsValid)
90          return BadRequest(ModelState);
```

```
91
92     // [Lấy userId từ JWT nếu cần]
93     // var userId = int.Parse(User.FindFirst
94                                     (ClaimTypes.NameIdentifier)!.Value);
95
96     var item = new Product
97     {
98         Name = dto.Name,
99         Price = dto.Price,
100        // UserId = userId,           // ← Nếu cần gán userId từ JWT
101        CreatedAt = DateTime.Now,    // ← Server tự set, không nhận từ client
102    };
103
104    _context.Products.Add(item);
105    await _context.SaveChangesAsync();
106
107    return Ok(new { message = "Tạo thành công.", id = item.Id });
108    // Hoặc trả 201 Created:
109    // return CreatedAtAction(nameof(GetById), new { id = item.Id }, item);
110
111
112    // =====
113    // PUT /api/products/{id}
114    // Cập nhật toàn bộ 1 bản ghi
115    // =====
116    [HttpPut("{id}")]
117    // [Authorize(Roles = "Admin")]
118    public async Task<IActionResult> Update(int id, [FromBody]
119        UpdateProductDTO dto)
120    {
121        if (!ModelState.IsValid)
122            return BadRequest(ModelState);
123
124        var item = await _context.Products.FindAsync(id);
125        if (item == null)
126            return NotFound($"Không tìm thấy Product với Id = {id}.");
127
128        // [Kiểm tra quyền - chỉ owner mới sửa được, nếu cần]
129        // var userId = int.Parse(User.FindFirst
130                                     (ClaimTypes.NameIdentifier)!.Value);
131        // if (item.UserId != userId) return Forbid();
132
133        // Cập nhật từng field từ DTO
134        item.Name = dto.Name;
135        item.Price = dto.Price;
```

```
134         // item.UpdatedAt = DateTime.Now; // ← Nếu có field UpdatedAt
135
136         _context.Products.Update(item);
137         await _context.SaveChangesAsync();
138
139         return Ok(new { message = "Cập nhật thành công." });
140     }
141
142
143     // =====
144     // PATCH /api/products/{id}
145     // Cập nhật 1 phần (chỉ field nào gửi lên mới đổi)
146     // =====
147     [HttpPatch("{id}")]
148     // [Authorize(Roles = "Admin")]
149     public async Task<IActionResult> PartialUpdate(int id, [FromBody] PatchProductDTO dto)
150     {
151         var item = await _context.Products.FindAsync(id);
152         if (item == null)
153             return NotFound($"Không tìm thấy Product với Id = {id}.");
154
155         // Chỉ update field nào client gửi lên (khác null)
156         if (dto.Name != null) item.Name = dto.Name;
157         if (dto.Price != null) item.Price = dto.Price.Value;
158
159         await _context.SaveChangesAsync();
160
161         return Ok(new { message = "Cập nhật một phần thành công." });
162     }
163
164
165     // =====
166     // DELETE /api/products/{id}
167     // Xóa 1 bản ghi
168     // =====
169     [HttpDelete("{id}")]
170     // [Authorize(Roles = "Admin")]
171     public async Task<IActionResult> Delete(int id)
172     {
173         var item = await _context.Products.FindAsync(id);
174         if (item == null)
175             return NotFound($"Không tìm thấy Product với Id = {id}.");
176
177         // [Kiểm tra quyền nếu cần]
178         // var userId = int.Parse(User.FindFirst
179         //     (ClaimTypes.NameIdentifier)!.Value);
180         // if (item.UserId != userId) return Forbid();
```

```
181         _context.Products.Remove(item);
182         await _context.SaveChangesAsync();
183
184         return Ok(new { message = "Xóa thành công." });
185     }
186
187
188     // =====
189     // GET /api/products/search?name=apple&minPrice=10&maxPrice=100
190     // Tìm kiếm + lọc (mở rộng thêm nếu cần)
191     // =====
192     [HttpGet("search")]
193     public async Task<IActionResult> Search(
194         [FromQuery] string? name,
195         [FromQuery] decimal? minPrice,
196         [FromQuery] decimal? maxPrice)
197     {
198         var query = _context.Products.AsQueryable();
199
200         if (!string.IsNullOrEmpty(name))
201             query = query.Where(p => p.Name.Contains(name));
202
203         if (minPrice.HasValue)
204             query = query.Where(p => p.Price >= minPrice.Value);
205
206         if (maxPrice.HasValue)
207             query = query.Where(p => p.Price <= maxPrice.Value);
208
209         var result = await query
210             .Select(p => new ProductResponseDTO
211             {
212                 Id = p.Id,
213                 Name = p.Name,
214                 Price = p.Price,
215             })
216             .ToListAsync();
217
218         return Ok(result);
219     }
220
221 }
222 }
223
224 // =====
225 // TEMPLATE DTOs - Copy và thay thế:
226 //   "Product" → tên entity của bạn
227 //   Thêm/bớt field cho phù hợp với Model thực tế
228 // =====
229
```

```
230 using System.ComponentModel.DataAnnotations;
231
232 namespace YourProject.DTOs
233 {
234     // =====
235     // RESPONSE DTO - Dùng khi TRẢ DỮ LIỆU về cho client
236     // Dùng trong: GET /api/products, GET /api/products/{id}
237     // Chỉ expose field nào client được phép thấy
238     // =====
239     public class ProductResponseDTO
240     {
241         public int Id { get; set; }
242         public string Name { get; set; } = null!;
243         public decimal Price { get; set; }
244         public DateTime CreatedAt { get; set; }
245         // Thêm field khác nếu cần trả về
246         // public string CategoryName { get; set; } // Nếu có include
247         Category
248     }
249
250     // =====
251     // CREATE DTO - Dùng khi client GỬI DỮ LIỆU LÊN để tạo mới
252     // Dùng trong: POST /api/products
253     // Chỉ nhận field nào client được phép tự nhập
254     // Không nhận: Id, CreatedAt, UserId, Status (server tự set)
255     // =====
256     public class CreateProductDTO
257     {
258         [Required(ErrorMessage = "Tên không được để trống.")]
259         [StringLength(100, ErrorMessage = "Tên không được vượt quá 100 ký
260         tự.")]
261         public string Name { get; set; } = null!;
262
263         [Required(ErrorMessage = "Giá không được để trống.")]
264         [Range(0.01, double.MaxValue, ErrorMessage = "Giá phải lớn hơn
265         0.")]
266         public decimal Price { get; set; }
267
268         // Thêm field khác nếu cần
269         // public int CategoryId { get; set; }
270         // public string Description { get; set; }
271     }
272
273     // =====
274     // UPDATE DTO - Dùng khi client GỬI DỮ LIỆU LÊN để cập nhật
275     // Dùng trong: PUT /api/products/{id}
276     // Thường giống CreateDTO, nhưng tách riêng để linh hoạt
```

```
276 // =====
277 public class UpdateProductDTO
278 {
279     [Required(ErrorMessage = "Tên không được để trống.")]
280     [StringLength(100, ErrorMessage = "Tên không được vượt quá 100 ký ➤
        tự.")]
281     public string Name { get; set; } = null!;
282
283     [Required(ErrorMessage = "Giá không được để trống.")]
284     [Range(0.01, double.MaxValue, ErrorMessage = "Giá phải lớn hơn ➤
        0.")]
285     public decimal Price { get; set; }
286 }
287
288
289 // =====
290 // PATCH DTO - Dùng khi client chỉ update MỘT PHẦN
291 // Dùng trong: PATCH /api/products/{id}
292 // Tất cả field đều nullable - field nào null = không đổi
293 // =====
294 public class PatchProductDTO
295 {
296     [StringLength(100)]
297     public string? Name { get; set; } // null = không đổi Name
298
299     [Range(0.01, double.MaxValue)]
300     public decimal? Price { get; set; } // null = không đổi Price
301 }
302 }
```